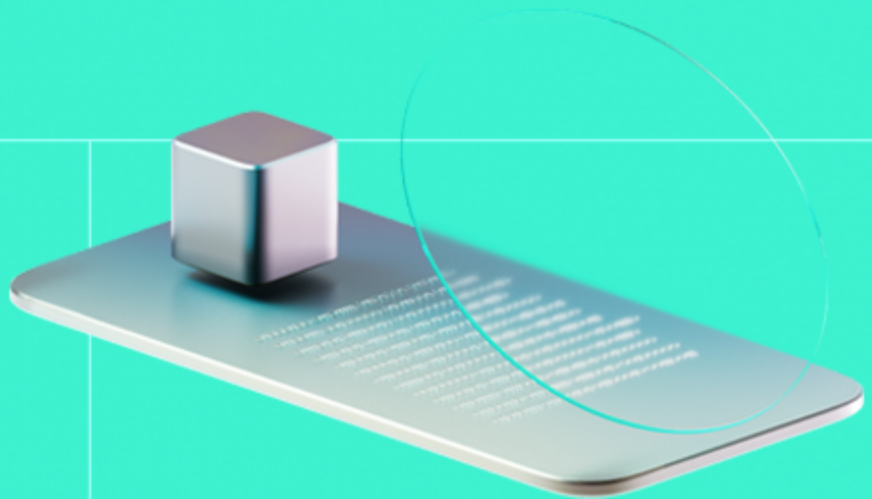




Smart Contract Code Review And Security Analysis Report

Customer: Push Chain

Date: 02/09/2026



We express our gratitude to the Push Chain team for the collaborative engagement that enabled the execution of this Smart Contract Security Assessment.

Push Chain is a Solana-side bridge gateway that locks user deposits in a PDA-controlled vault and releases them via TSS-signed outbound operations (withdraw, execute, revert, rescue), enabling bidirectional asset and payload bridging between Solana and Push Chain.

Document

Name	Smart Contract Code Review and Security Analysis Report for Push Chain
Audited By	Felipe Donato, Seher Saylik
Approved By	Ivan Bondar
Website	https://push.org
Changelog	10/04/2026 - Preliminary Report
	19/05/2026 - Remediation Report
	02/09/2026 - Final Report
Platform	SVM
Language	Rust
Tags	Bridge, Cross-chain, Vault
Methodology	https://docs.hacken.io/methodologies/smart-contracts

Review

Scope

Repository	https://github.com/pushchain/push-chain-gateway-contracts/tree/audit-main/contracts/svm-gateway
Initial Commit	93db3c8e66cc1399ee74fc58339440e78321ae06
Final Commit	d637de5b4bf4aa76837f9084c676817443507f94



Audit Summary

The system users should acknowledge all the risks summed up in the risks section of the report

20	16	2	2
Total Findings	Resolved	Accepted	Mitigated

Findings by Severity

Severity	Count
Critical	0
High	0
Medium	7
Low	3

Vulnerability	Severity	Status
F-2026-15625 - Wrong Gas Fee Reimbursement Source In finalize_universal_tx Breaks The 1:1 Backing Invariant	Medium	Mitigated
F-2026-15678 - Missing Per-Slot USD Cap in FUNDS Route Leads To Uncontrolled Deposit Throughput	Medium	Accepted
F-2026-15696 - Unsigned revert_msg Allows Forged Revert Data in Cross-Chain Events	Medium	Fixed
F-2026-15697 - Insufficient Privilege Separation in PauseAction Allows Pauser to Unpause the Gateway	Medium	Fixed
F-2026-15702 - Missing Canonical ATA Enforcement on Vault Token Accounts Causes SPL Liquidity Fragmentation	Medium	Fixed
F-2026-15755 - Inconsistent Pause Constraints Across Admin Functions	Medium	Fixed
F-2026-18980 - [Dual Defense] Missing Post-CPI Validation Allows a Destination Program to Persist Authority Mutations on the CEA and Its ATA	Medium	Fixed
F-2026-15624 - Excessive Pyth Oracle Staleness Window Enables Rate Limit Bypass	Low	Fixed
F-2026-15699 - set_token_rate_limit Unconditionally Resets Epoch Usage on Threshold Update	Low	Fixed
F-2026-15701 - limit_threshold = 0 Causes Token-Level DoS Instead of Disabling Rate Limiting	Low	Fixed
F-2026-15649 - Dead Code in send_tx_with_gas_route Zero-Amount Guard	Info	Fixed
F-2026-15654 - Misleading Function and Parameter Naming in Instant Route Handler	Info	Accepted
F-2026-15705 - Unprotected Initialization Allows First-Caller Takeover	Info	Fixed



Vulnerability	Severity	Status
F-2026-15754 - Redundant Payload Validation in Funds Route Handler	Info	Fixed
F-2026-15756 - Unbounded Protocol Fee	Info	Mitigated
F-2026-15757 - Lack of Two-Step Admin Transfer	Info	Fixed
F-2026-15930 - Redundant Oracle Price Calculation in Gas Route Deposit	Info	Fixed
F-2026-15948 - PDA Bumps Re-derived Instead of Using Stored Values in Finalize Instruction	Info	Fixed
F-2026-15957 - Missing Mint Authority Validation in set_token_rate_limit Allows Whitelisting Tokens with Unsafe Authorities	Info	Fixed
F-2026-18981 - Rate Limiting Scope Is Not Documented in the Threat Model	Info	Fixed

Documentation quality

- Functional requirements are provided.
- Technical description is provided.

Code quality

- Some compute-inefficient patterns were identified, such as redundant oracle calls or redundant validations.
- The development environment is configured.

Test coverage

Code coverage of the project is **95.4%** (line coverage).

- Deployment, initialization, and admin configuration flows are fully covered.
- Positive and negative cases are covered across deposit, withdraw, revert, and rescue paths, including replay protection, pause enforcement, zero-amount rejection, and insufficient balance checks.
- Multi-user interactions involving SPL token deposits into non-canonical vault accounts are covered via PoC tests.
- Execute-mode CPI paths with TSS signature verification are covered through the test-counter integration tests.
- Uncovered lines (4.6%) are primarily error branches in initialize.rs and edge cases in pricing.rs and validation.rs.

Table of Contents

System Overview	7
Privileged Roles	8
Potential Risks	9
Findings	9
Vulnerability Details	10
Disclaimers	62
Appendix 1. Definitions	63
Severities	63
Potential Risks	63
Appendix 2. Scope	64
Appendix 3. Additional Valuables	66

System Overview

The Push Chain SVM Gateway is the Solana-side bridge gateway for asset movement between Solana and Push Chain. It accepts inbound deposits on Solana, escrows assets under protocol-controlled accounts, and later processes outbound withdrawals, finalize operations, reverts, and rescues based on TSS authorization.

Architecture

The gateway maintains two core custody accounts:

- **Vault (`vault_sol`)** holds bridged native SOL and serves as the authority for SPL token vault accounts.
- **Fee Vault (`fee_vault`)** holds protocol fees collected from inbound deposits and is intended to fund relayer reimbursements without consuming bridge collateral.

Additional protocol state includes:

- **Config:** stores admin and pauser authorities, pause state, protocol fee settings, USD cap parameters, and Pyth oracle configuration.
- **TssPda:** stores the active TSS Ethereum address and chain ID used for secp256k1 verification on outbound operations.
- **CEA:** a per-user PDA used as a persistent execution identity for outbound CPI and staged execution flows.
- **ExecutedSubTx:** replay-protection state keyed by `sub_tx_id`.
- **RateLimitConfig:** stores the per-slot USD cap and epoch duration for inbound rate limiting.
- **TokenRateLimit:** stores per-token epoch-based rate-limit state.

Inbound Flow

Users initiate deposits through `send_universal_tx`. The gateway collects the protocol fee, classifies the request by route type, applies route-specific controls, transfers assets into protocol custody, and emits a `UniversalTx` event for downstream processing.

- **GAS / GAS_AND_PAYLOAD:** native SOL route with USD-denominated checks using the configured Pyth SOL/USD feed.
- **FUNDS / FUNDS_AND_PAYLOAD:** standard bridge route for native SOL or SPL tokens with token-based epoch rate limiting.

Outbound Flow

All outbound operations require a valid TSS secp256k1 signature. The gateway supports four outbound paths:

- **Withdraw (`withdraw`)** releases assets from protocol custody to a Solana recipient.
- **Finalize (`finalize_universal_tx`)** stages assets to a user-specific CEA, then transfers, executes CPI, or routes assets back into the gateway.



- **Revert** (`revert_universal_tx`) returns assets when a cross-chain operation must be canceled or rolled back.
- **Rescue** (`rescue_funds`) provides an operator-controlled recovery path for exceptional cases.

Privileged Roles

The **admin** (`config.admin`) holds the highest privilege. It can update the TSS address, rotate admin and pauser authorities, configure the Pyth oracle feed, set USD caps and rate limits, set the protocol fee, and pause or unpause the gateway. All admin actions take effect immediately with no timelock.

The **pauser** (`config.pauser`) can pause and unpause the gateway. The admin can also pause and unpause, meaning the pauser cannot enforce a pause against the admin's will.

The **TSS** is the sole authorization mechanism for all outbound operations. It produces ECDSA secp256k1 signatures verified on-chain. No funds can leave the vault without a valid TSS signature.

Potential Risks

- **TSS Compromise:** The TSS threshold signature scheme is the sole authorization mechanism for all outbound fund releases (withdraw, execute, revert, rescue). A compromised TSS can drain the entire vault.
- **Off-Chain Liveness Dependency:** Users who deposit funds via `send_universal_tx` have no on-chain mechanism to cancel or reclaim their deposit if the off-chain infrastructure (Universal Validators, TSS) becomes unresponsive.
- **Oracle Dependency:** Inbound gas route deposits rely on a single Pyth SOL/USD price feed for USD cap enforcement. If the Pyth feed becomes unavailable, stale, or manipulated, all gas route deposits are either blocked or processed with incorrect price data.
- **Upgradeable Program:** The program is upgradeable via the Solana upgrade authority. A compromised upgrade authority can replace the program logic entirely, overriding all on-chain security controls including TSS verification and replay protection.
- **Centralization Risk:** All privileged operations (authority rotation, TSS update, oracle swap, rate limit configuration, pause/unpause) are controlled by a single admin signer with no timelock, multisig enforcement, or two-step transfer. A compromised admin key can reconfigure the entire protocol in a single block.
- **Off-Chain Token Whitelisting Assumption:** If the protocol opts for off-chain validation of token mint authorities instead of on-chain enforcement, the security of vault-held SPL tokens depends on an operational trust assumption outside the audit scope. Tokens with active freeze or mint authorities carry risks described in findings section.

Findings

Vulnerability Details

[F-2026-15625](#) - Wrong Gas Fee Reimbursement Source In `finalize_universal_tx` Breaks The 1:1 Backing Invariant - Medium

Description:

The protocol maintains two separate vaults by design: `vault_sol` (bridge vault - holds native SOL backing for bridged native SOL positions) and `fee_vault` (protocol fee pool - funded by inbound protocol fees, used for relayer reimbursement).

The `rescue_funds` and `revert_universal_tx` paths correctly reimburse relayers from `fee_vault`. However, `finalize_universal_tx` reimburses from `vault_sol` via `transfer_gas_fee_to_caller()`. Each `finalize` therefore debits `amount + gas_fee` from the vault while inbound deposits only credit `amount`. The `gas_fee` portion has no matching inflow into `vault_sol`, breaking the vault's intended 1:1 backing invariant.

`instructions/execute.rs` - `stage_assets_to_cea()`:

```
fn stage_assets_to_cea(
    ctx: &Context<FinalizeUniversalTx>,
    request: &FinalizeRequestContext,
    amount: u64,
    gas_fee: u64,
    vault_seeds: &[u8],
) -> Result<()> {
    if request.is_native {
        pda_system_transfer(
            &ctx.accounts.vault_sol.to_account_info(), // bridge vault
            &ctx.accounts.ce_a_authority.to_account_info(),
            &ctx.accounts.system_program.to_account_info(),
            amount,
            vault_seeds,
        )?;
    } else {
        process_spl_vault_to_cea_transfer(ctx, amount, vault_seeds)?;
    }

    // Relayer reimbursement pulled from vault_sol (bridge assets)
    crate::utils::transfer_gas_fee_to_caller(
        &ctx.accounts.vault_sol.to_account_info(), // <-- Should be fee_v
    ault
```

```

        &ctx.accounts.caller.to_account_info(),
        &ctx.accounts.system_program.to_account_info(),
        gas_fee,
        ctx.accounts.config.vault_bump,
    )
}

```

Compare with the correct implementation used by `rescue_funds` and `revert_universal_tx`:

```

// --> Gas fee pulled from fee_vault (protocol fee pool)
reimburse_relayer_from_fee_vault(
    &ctx.accounts.fee_vault,
    &ctx.accounts.caller.to_account_info(),
    sub_tx_id,
    gas_fee,
)?;

```

- **Native SOL vault undercollateralization:** every finalize transaction systematically reduces the vault's ability to honor future native SOL withdrawals, reverts, and rescues.
- **Cumulative deficit:** the shortfall increases monotonically under normal operation with no automatic replenishment mechanism in the finalize flow.
- **Cross-asset coupling:** SPL-path finalizations also drain `vault_sol` for native SOL reimbursement, meaning SPL outbound volume degrades native SOL solvency.
- **No attacker required:** normal protocol operation causes the drain. The TSS signs legitimate reimbursement values, and every legitimate finalize contributes to the deficit.

Status: Mitigated

Classification

Impact Rate: 2/5

Likelihood Rate: 2/5

Exploitability: Independent

Complexity: Simple

Severity: Medium

Recommendations

Remediation:

Replace the `transfer_gas_fee_to_caller()` call in `stage_assets_to_cea()` with `reimburse_relayer_from_fee_vault()`, so relayer reimbursement is sourced from the protocol fee pool rather than the bridge vault. This aligns `finalize_universal_tx()` with the reimbursement model already used by `revert_universal_tx()` and `rescue_funds()`.

```
fn stage_assets_to_cea(
    ctx: &Context<FinalizeUniversalTx>,
    request: &FinalizeRequestContext,
    sub_tx_id: [u8; 32],
    amount: u64,
    gas_fee: u64,
    vault_seeds: &[&[u8]],
) -> Result<()> {
    if request.is_native {
        pda_system_transfer(
            &ctx.accounts.vault_sol.to_account_info(),
            &ctx.accounts.ce_a_authority.to_account_info(),
            &ctx.accounts.system_program.to_account_info(),
            amount,
            vault_seeds,
        )?;
    } else {
        process_spl_vault_to_cea_transfer(ctx, amount, vault_seeds)?;
    }
    // --> Gas fee from fee_vault (protocol fee pool), not bridge vault
    reimburse_relayer_from_fee_vault(
        &ctx.accounts.fee_vault,
        &ctx.accounts.caller.to_account_info(),
        sub_tx_id,
        gas_fee,
    )
}
```

This change also requires adding `fee_vault: Account<'info, FeeVault>` to the `FinalizeUniversalTx` accounts struct and passing `sub_tx_id` into `stage_assets_to_cea()`, since the finalize path is not currently integrated with the fee-vault reimbursement flow.

Resolution:

Mitigated in commit `762ab26`,

The fix addresses the reimbursement-accuracy component by computing `gas_used` on-chain, retaining surplus as `gas_to_refund`, and extending the emitted event so Push-side infrastructure can process the refund.

However, the original reimbursement-source issue remains unresolved:

`settle_relayer_gas_cost()` still reimburses from `vault_sol`, and `fee_vault` is still

not part of the `FinalizeUniversalTx` path. As a result, the source-of-funds behavior described in the finding remains unchanged.

The cross-asset coupling also persists, since on the scoped SPL `FUNDS` path inbound deposits add no native SOL to `vault_sol`, while SPL finalize still consumes native SOL from `vault_sol` through `gas_used`.

Additionally, the new `gas_used` model depends on documented but unenforced operational assumptions, including the fixed `SIGNATURE_FEE_LAMPORTS` value and the single-signature / single-fee-payer model, while the `gas_to_refund` flow remains dependent on off-chain Push-side processing. The finding, therefore, remains open on the reimbursement-source axis.

Evidences

Wrong Gas Fee Reimbursement Source In `finalize_universal_tx` Breaks The 1:1 Backing Invariant

Reproduce:

Steps to Reproduce:

- Initialize the gateway and fund the vault with SOL.
- TSS signs a `finalize_universal_tx` (instruction_id=1, withdraw mode) for 1 SOL (`amount = 1_000_000_000 lamports`) with a relayer reimbursement of 0.01 SOL (`gas_fee = 10_000_000 lamports`).
- Relayer submits the transaction. `stage_assets_to_cea` transfers 1 SOL to the CEA and 0.01 SOL to the relayer - both from `vault_sol`.
- Vault balance is now reduced by `gas_fee` beyond the user withdrawal amount, creating a deficit relative to the expected 1:1 native SOL backing model.
- Repeat across N transactions. Each finalize leaks `gas_fee` lamports from the bridge vault. After sufficient volume, the native SOL vault may be unable to fully cover remaining native SOL bridged positions.

Results:

PUSH-GW-001: Bridge Vault 1:1 Invariant Violation

```
||-----||
|| PUSH-GW-001: Bridge Vault Invariant Violation PoC ||
||-----||
|| Withdraw amount:           1,000,000,000 lamports ||
|| gas_fee:                   10,000,000 lamports ||
||-----||
|| Vault drained:             1,010,000,000 lamports ||
|| Recipient received:       1,000,000,000 lamports ||
|| Fee vault change:          0 lamports ||
```

```

||-----||
|| DEFICIT (gas_fee):           10,000,000 lamports ||
||-----||
|| vault drained amount + gas_fee, NOT just amount ||
|| fee_vault untouched – reimbursement came from ||
|| bridge assets, not protocol fee pool           ||
||-----||

```

✓ PUSH-GW-001: Every finalize_universal_tx leaks gas_fee from bridge vault (441ms)

```

||-----||
|| PUSH-GW-001: Cumulative Drain Proof             ||
||-----||
|| Iterations:                               5      ||
|| Per-tx withdraw:           100,000,000 lamports ||
|| Per-tx gas_fee:            5,000,000 lamports  ||
||-----||
|| Total withdrawn:           500,000,000 lamports ||
|| Total vault drain:         525,000,000 lamports ||
|| Total leaked:              25,000,000 lamports ||
|| Expected leak:             25,000,000 lamports ||
|| Fee vault change:          0 lamports          ||
||-----||
|| Leak scales linearly with tx volume.           ||
|| Fee vault never touched. Bridge vault insolvent. ||
||-----||

```

PUSH-GW-001: Cross-Asset Coupling - SPL Finalize Drains Native SOL

```

||-----||
|| PUSH-GW-001: Cross-Asset Coupling PoC (Step 1) ||
|| SPL Deposit Phase                               ||
||-----||
|| SPL deposit:      100 USDT                      ||
|| vault_sol delta:                                0 lamports ||
|| vault ATA delta:  100000000 tokens              ||
||-----||
|| Deposit succeeded: USDT entered vault ATA.      ||
|| But ZERO native SOL entered vault_sol.          ||
||-----||

```

✓ SPL deposit contributes zero native SOL to vault_sol (407ms)

```

||-----||
|| PUSH-GW-001: Cross-Asset Coupling PoC (Step 2) ||
|| SPL Finalize Phase                               ||
||-----||

```

```
|| SPL withdraw: 50 USDT ||
|| gas_fee: 10000000 lamports ||
||-----||
|| vault_sol drain: 10000000 lamports ||
|| fee_vault delta: 0 lamports ||
||-----||
|| On this SPL FUNDS path, zero native SOL entered ||
|| vault_sol, but native SOL left for gas_fee. ||
||-----||

✓ SPL finalize drains native SOL from vault_sol for gas_fee (814ms)
```

Files:

poc.test.ts

poc4.test.ts

[F-2026-15678](#) - Missing Per-Slot USD Cap in FUNDS Route Leads To Uncontrolled Deposit Throughput - Medium

Description:

The deposit flow uses two distinct rate-limiting mechanisms: a per-slot USD cap (`check_block_usd_cap`) that limits total USD throughput per Solana slot, and a per-token epoch-based rate limit (`validate_token_and_consume_rate_limit`) that limits volume per token over time.

However, these two mechanisms are applied on different routes:

Route	Per-slot USD cap	Token epoch rate limit
Gas / GasAndPayload	[OK] <code>check_block_usd_cap</code>	[X] Not applied
Funds / FundsAndPayload	[X] Not applied	[OK] <code>validate_token_and_consume_rate_limit</code>

`instructions/deposit.rs` - `handle_native_funds_route()`:

```
fn handle_native_funds_route(...) -> Result<()> {
    // Gas excess goes through gas route (has block USD cap)
    let gas_amount = native_amount.saturating_sub(req.amount);
    if gas_amount > 0 {
        send_tx_with_gas_route(ctx, TxType::Gas, gas_amount, ...)?; // block
    }

    // FUNDS amount only checked against token rate limit - no block USD cap
    validate_token_and_consume_rate_limit(
        &mut ctx.accounts.token_rate_limit,
        Pubkey::default(),
        req.amount as u128,
        &ctx.accounts.rate_limit_config,
    );
    // ... transfer
}
```

Compare with the gas route which applies the per-slot cap:

```
fn send_tx_with_gas_route(...) -> Result<()> {
    check_usd_caps(&ctx.accounts.config, gas_amount, &ctx.accounts.price_update)?;
    let price_data = calculate_sol_price(&ctx.accounts.price_update)?;
    let usd_amount = calculate_usd_amount(gas_amount, &price_data)?;
    check_block_usd_cap(&mut ctx.accounts.rate_limit_config, usd_amount)?;
}
```

```
// ... transfer
}
```

- If the per-slot USD cap is intended as a global inbound throughput control, FUNDS deposits are not subject to that control and are instead governed only by the token epoch-based limit.
- For native `FundsAndPayload`, the bypass is partial: the gas-excess leg is block-capped, but the funds leg is not. For pure `Funds` (native or SPL), the bypass is complete. The same gap exists in `handle_spl_funds_route()`, which applies `validate_token_and_consume_rate_limit()` but does not invoke `check_block_usd_cap()`.
- Because the epoch-based limit operates over a longer time window, single-slot burst volume remains unconstrained on the FUNDS path.

Assets:

- deposit.rs [<https://github.com/pushchain/push-chain-gateway-contracts/tree/audit-main/contracts/svm-gateway>]

Status:

Accepted

Classification

Impact Rate:	4/5
Likelihood Rate:	3/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Medium

Recommendations

Remediation:

Apply `check_block_usd_cap` to the FUNDS route for native SOL, where the existing Pyth price feed already provides USD conversion:

```
fn handle_native_funds_route(...) -> Result<(),> {
    // ... existing gas route handling ...

    validate_token_and_consume_rate_limit(...)?;

    // Add: block USD cap for funds amount
    let price_data = calculate_sol_price(&ctx.accounts.price_update)?;
    let usd_amount = calculate_usd_amount(req.amount, &price_data)?;
    check_block_usd_cap(&mut ctx.accounts.rate_limit_config, usd_amount)?;
```

```
// ... transfer  
}
```

For SPL tokens, the current codebase does not have per-token USD pricing in the deposit path. Applying the per-slot cap to SPL FUNDS would require either adding per-token price feeds or defining a token-to-USD conversion mechanism. Alternatively, if per-slot enforcement for SPL is not feasible, document that SPL FUNDS deposits rely exclusively on the epoch-based token limit as the throughput control.

Resolution:**Accepted.**

The Push Chain team stated that:

This was intended. Block USD cap for instant route, and epoch rate limiting for FUNDS route

[F-2026-15696](#) - Unsigned revert_msg Allows Forged Revert Data in Cross-Chain Events - Medium

Description:

The `revert_universal_tx` function accepts a `RevertInstructions` struct containing `revert_recipient` and `revert_msg`. However, the TSS-signed message for `instruction_id = 3` covers only `sub_tx_id`, `universal_tx_id`, optional `mint`, `recipient`, and `gas_fee`; it does not include `revert_msg`.

The function later emits `RevertUniversalTx`, which includes `revert_instruction.clone()`. Because `RevertInstructions` contains `revert_msg`, that field is carried in the emitted cross-chain revert event even though it is outside the TSS-signed payload. This creates a signature-coverage gap in the revert data surface.

```
emit!(crate::state::RevertUniversalTx {
    sub_tx_id,
    universal_tx_id,
    revert_recipient: revert_instruction.revert_recipient,
    token: ...,
    amount,
    revert_instruction: revert_instruction.clone(), // <-- contains unsigned
    revert_msg
});
```

`revert_universal_tx` constructs the signed message without `revert_msg`:

```
// TSS message: instruction_id=3 || amount || [sub_tx_id, universal_tx_id, (mint,) recipient, gas_fee]
//
^^^ NO revert_msg
let additional: [&[u8]; 4] = [&sub_tx_id, &universal_tx_id, &recipient_bytes,
&gas_fee_buf];
validate_message(&mut ctx.accounts.tss_pda, 3, Some(amount), &additional, ...
)?;
```

- A relay can supply arbitrary `revert_msg` content in the emitted cross-chain revert event without TSS authorization. On-chain fund movement is unaffected because `amount`, `recipient`, and `token` remain signed.
- if downstream components consume `revert_msg` as authenticated bridge metadata, a relay can inject forged revert data into the cross-chain event stream.

- If `revert_msg` is intended to be advisory only, that trust model should be documented explicitly.

Assets:

- `revert.rs` [<https://github.com/pushchain/push-chain-gateway-contracts/tree/audit-main/contracts/svm-gateway>]

Status:**Fixed**

Classification**Impact Rate:** 3/5**Likelihood Rate:** 3/5**Exploitability:** Independent**Complexity:** Simple**Severity:** **Medium**

Recommendations**Remediation:**

Include a hash of `revert_msg` in the TSS-signed payload, or explicitly document that `revert_msg` is untrusted advisory data and ensure no downstream consumer treats it as authenticated.

```
let revert_msg_hash = hash(&revert_instruction.revert_msg).to_bytes();
let additional: [&[u8]; 5] = [
    &sub_tx_id, &universal_tx_id, &recipient_bytes, &gas_fee_buf, &revert_msg
    _hash
];
```

Resolution:**Fixed in commit** `92113b5`,

Added `keccak256(revert_msg)` to the TSS-signed revert payload so `revert_msg` is now authenticated and cannot be forged in emitted cross-chain revert events.

[F-2026-15697](#) - Insufficient Privilege Separation in PauseAction Allows Pauser to Unpause the Gateway - Medium

Description:

Both `pause()` and `unpause()` use the same `PauseAction` accounts struct, which authorizes either `config.pauser` or `config.admin`. This means the pauser role can both pause and unpause the gateway.

In standard bridge designs, the pauser is a lower-privilege role (often a monitoring bot or multisig signer) that can trigger an emergency halt. Unpause is typically restricted to admin-only because re-enabling the system after a security incident requires higher assurance that the underlying issue is resolved.

```
#[derive(Accounts)]
pub struct PauseAction<'info> {
    #[account(
        mut,
        seeds = [CONFIG_SEED],
        bump = config.bump,
        constraint = config.pauser == pauser.key() || config.admin == pauser.
key()
        @ GatewayError::Unauthorized
    )]
    pub config: Account<'info, Config>,
    pub pauser: Signer<'info>,
}

pub fn unpause(ctx: Context<PauseAction>) -> Result<()> {
    ctx.accounts.config.paused = false; // Uses PauseAction - pauser can ca
    ll this
    Ok(())
}
```

A pauser can reverse an admin-initiated emergency halt without admin approval. This breaks the intended separation between emergency stop authority and recovery authority, allowing the pauser role to overrule incident containment decisions.

Assets:

- `admin.rs` [<https://github.com/pushchain/push-chain-gateway-contracts/tree/audit-main/contracts/svm-gateway>]

Status:

Fixed

Classification

Impact Rate:	3/5
Likelihood Rate:	2/5
Exploitability:	Semi-Dependent
Complexity:	Simple
Severity:	Medium

Recommendations

Remediation: Create a separate accounts struct for `unpause()` that only authorizes `config.admin`:

```
#[derive(Accounts)]
pub struct UnpauseAction<'info> {
    #[account(
        mut,
        seeds = [CONFIG_SEED],
        bump = config.bump,
        constraint = config.admin == admin.key() @ GatewayError::Unauthorized
    )]
    pub config: Account<'info, Config>,
    pub admin: Signer<'info>,
}

pub fn unpause(ctx: Context<UnpauseAction>) -> Result<()> {
    ctx.accounts.config.paused = false;
    Ok(())
}
```

Resolution: Fixed in commit `0a0c9c1`.

Split pause and unpause authorization by introducing `UnpauseAction`; pauser can pause only, while unpause now requires `config.admin`.

[F-2026-15702](#) - Missing Canonical ATA Enforcement on Vault Token Accounts Causes SPL Liquidity Fragmentation - Medium

Description:

The program does not enforce a canonical vault ATA for a mint. SPL collateral for the same mint can be deposited into multiple vault-owned token accounts, fragmenting custody. The deposit path checks only that `gateway_token_account` has `owner == vault PDA` and `mint == token`, not that it is the canonical ATA for the `(vault, mint)` pair:

```
// deposit.rs - deposit_spl_to_vault()
let parsed = parse_token_account(&gateway_token_account.to_account_info());
require!(parsed.owner == ctx.accounts.vault.key(), GatewayError::InvalidOwner);
require!(parsed.mint == token, GatewayError::InvalidMint);
// No check that gateway_token_account == ATA(vault, mint)
```

The same loose pattern exists on the outbound side. In `FinalizeUniversalTx`, the `vault_ata` uses `token::authority = vault_sol` (any vault-owned token account passes). In `revert` and `rescue`, the `token_vault` is validated with `token_vault.owner == vault.key()` and `token_vault.mint == mint_key` - again, not canonically derived.

Compare with how other paths in the codebase enforce canonical ATAs:

```
// execute.rs - CEA ATA validation
let expected_cea_ata = spl_associated_token_account::get_associated_token_address(
    &ctx.accounts.cea_authority.key(), &mint.key(),
);
require!(cea_ata.key() == expected_cea_ata, GatewayError::InvalidAccount);

// withdraw.rs - recipient ATA validation
let expected_recipient_ata =
    spl_associated_token_account::get_associated_token_address(&target, &token_mint.key());
require!(recipient_ata.key() == expected_recipient_ata, GatewayError::InvalidAccount);
```

Both outbound CEA and recipient paths derive and enforce the canonical ATA. The vault token account is the only one that doesn't.

While outbound paths also accept non-canonical vault-owned accounts, any infrastructure that assumes a single canonical vault ATA for a mint can experience accounting desynchronization or attempt to source liquidity from an unexpectedly underfunded vault account.

- **Liquidity fragmentation:** SPL liquidity controlled by the vault PDA can be distributed across multiple token accounts for the same mint.
- **Divergence from expected custody location:** if relayers or accounting systems assume the canonical vault ATA, withdrawals, reverts, and rescues may select an unexpectedly underfunded vault account even though the protocol controls enough tokens in aggregate.
- **Low-cost griefing:** an attacker can induce this state while performing an otherwise valid bridge deposit, paying only the additional cost of creating a non-canonical vault-owned token account.
- **Multi-path exposure:** the same loose validation exists on deposit, finalize, revert, and rescue paths.

Assets:

- execute.rs [https://github.com/pushchain/push-chain-gateway-contracts/tree/audit-main/contracts/svm-gateway]
- deposit.rs [https://github.com/pushchain/push-chain-gateway-contracts/tree/audit-main/contracts/svm-gateway]
- revert.rs [https://github.com/pushchain/push-chain-gateway-contracts/tree/audit-main/contracts/svm-gateway]
- rescue.rs [https://github.com/pushchain/push-chain-gateway-contracts/tree/audit-main/contracts/svm-gateway]

Status: Fixed

Classification

Impact Rate: 3/5

Likelihood Rate: 3/5

Exploitability: Independent

Complexity: Simple

Severity: Medium

Recommendations

Remediation: Enforce the canonical ATA on all paths that interact with vault token accounts:

```
// Derive and enforce canonical ATA
let expected_vault_ata = spl_associated_token_account::get_associated_token_a
ddress(
    &ctx.accounts.vault.key(),
    &token,
);
```

```
require!(
  gateway_token_account.key() == expected_vault_ata,
  GatewayError::InvalidAccount
);
```

Apply this check consistently in:

- `deposit_spl_to_vault()` for `gateway_token_account`
- `FinalizeUniversalTx` for `vault_ata` (add canonical ATA derivation in addition to authority/mint validation)
- `RevertUniversalTx` for `token_vault`
- `RescueFunds` for `token_vault`

If multiple vault-owned token accounts are intentional, the exact receiving account must be tracked in authenticated state and surfaced in events so off-chain infrastructure can reconcile balances across accounts.

Resolution:

Fixed in commit `2620129`.

The canonical ATA enforcement was added for vault token accounts across deposit, finalize, revert, and rescue flows; non-canonical vault-owned token accounts are now rejected.

Evidences

Missing Canonical ATA Enforcement on Vault Token Accounts Causes SPL Liquidity Fragmentation

Reproduce:

Steps To Reproduce:

Anyone can create a non-canonical token account with `owner = vault PDA` - SPL token's `InitializeAccount` does not require the owner to sign. The attacker then:

1. Creates a rogue token account with `owner = vault PDA` for a supported mint (~0.002 SOL rent).
2. Deposits SPL tokens into it via `send_universal_tx`. All deposit validation passes.
3. The deposit succeeds, but the source-side collateral is placed into a non-canonical vault-owned token account rather than the canonical vault ATA.

Results:

PUSH-GW-008: SPL Liquidity Fragmentation via Non-Canonical Vault ATA

[SETUP] Rogue vault-owned token account created

Rogue account: `HNbppHsR6BqL53xohR4RN364FpcS89Ek1BKTzgnH2PGH`

Owner: `GhhFnhDKE6ZqXqMSjDQoDZr6BAK4Gq5oBtNqk71D4tbN (vault PDA)`

Mint: 2XBCYZuhcu8mdGwmJPQDHPYkbDN8SS5hWTggMvfMNriR
Canonical ATA: GMkY4scC3a6J6vu7vTdHWG8y66jrSroYqnidyHxJqaQr
Rogue != Canonical: true

```
||  
|| PUSH-GW-008: SPL Liquidity Fragmentation PoC ||  
||  
|| Deposit amount: 100000000 tokens ||  
||  
|| Canonical ATA delta: 0 tokens ||  
|| Rogue account delta: 100000000 tokens ||  
|| Attacker balance: -100000000 tokens ||  
||  
|| Canonical ATA received NOTHING. ||  
|| Rogue vault-owned account received the full deposit. ||  
|| Vault PDA controls both accounts for the same mint. ||  
|| Custody is fragmented across non-canonical accounts. ||  
||
```

✓ PUSH-GW-008: Deposit into non-canonical vault account fragments SPL liquidity (419ms)

Files:

poc2.test.ts

F-2026-15755 - Inconsistent Pause Constraints Across Admin

Functions - Medium

Description:

Admin functions use two different account structs with inconsistent pause behavior. Some admin operations are blocked during paused state while others are not, despite all being restricted to the same `config.admin` signer.

Blocked during pause:

- `set_caps_usd()`
- `set_pyth_price_feed()`
- `set_pyth_confidence_threshold()`
- `set_block_usd_cap()`
- `update_epoch_duration()`
- `set_token_rate_limit()`

Allowed during pause:

- `set_authorities()`
- `set_protocol_fee()`
- `init_tss() / update_tss()`
- `pause() / unpause()`

The pause constraint on the first group serves no practical purpose because the admin can always call `unpause` first — the `PauseAction` struct allows both admin and pauser to unpause. The admin workflow to change a blocked parameter while paused is simply: `unpause` → `set_caps_usd` → `pause`, all in sequential transactions. No other role can prevent this since the admin has unconditional unpause authority.

The affected functions (`set_caps_usd`, `set_pyth_price_feed`, etc.) only control inbound deposit parameters, and `send_universal_tx` is itself blocked during pause. So even if these setters were allowed during pause, there would be no user-facing impact until the protocol is unpaused.

During an emergency where the admin pauses the protocol to respond to a misconfigured oracle feed, manipulated rate limits, or incorrect USD caps, the admin cannot directly fix the root cause while the protocol remains safely paused. Instead, the admin must temporarily unpause the protocol to apply the fix, then re-pause.

Assets:

- `admin.rs` [<https://github.com/pushchain/push-chain-gateway-contracts/tree/audit-main/contracts/svm-gateway>]

Status:

Fixed

Classification

Impact Rate:	3/5
Likelihood Rate:	5/5
Exploitability:	Dependent
Complexity:	Simple
Severity:	Medium

Recommendations

Remediation: Align all admin functions to the same pattern. Remove the `!config.paused` constraint from `AdminAction`, `RateLimitConfigAction`, and `TokenRateLimitAction` to match the other admin structs.

Resolution: **Fixed in commit** `0564ef3`.

The fix was applied by removing the pause guards from the admin setters, so all admin operations are now uniformly allowed during pause.

[F-2026-18980](#) - [Dual Defense] Missing Post-CPI Validation Allows a Destination Program to Persist Authority Mutations on the CEA and Its ATA - Medium

Description:

In execute mode, `dispatch_finalize_action` promotes the CEA to a CPI signer for the caller specified destination program:

```
let is_signer = account.pubkey == cea_key;
...
invoke_signed(&cpi_ix, ctx.remaining_accounts, &[cea_seeds]);
Ok(())
```

The only constraint on that program is `require!(destination_program.executable)`. The client has confirmed there is no allowlist, so any executable Solana program is a valid target.

Signer privilege propagates into the target's nested CPIs. Control returns from `invoke_signed` straight to `Ok(())`, and no CEA side state is re-read afterwards. Pre-CPI validation of the CEA ATA covers address, mint and owner, but not `delegate`, `delegated_amount` or `close_authority`.

A target can therefore leave persistent authority state behind via `spl_token::Approve`, `spl_token::SetAuthority(AccountOwner)`, or `system_instruction::assign`.

Funds at risk are user principal at rest, bounded to the CEA ATA of a user who executed against the target. The vault is not reachable, so there is no path to protocol insolvency or to users who never interacted.

Variant	Effect	Reaches future balances
<code>Approve</code>	Unbounded standing delegate on the CEA ATA	Yes
<code>SetAuthority</code>	Takes the current ATA balance, then all later staging reverts at the owner check before any vault transfer	No
<code>assign</code>	Every finalize path for that <code>push_account</code> fails; native SOL on the CEA is stranded	No

Only the delegate variant multiplies. The user authorises one call for one amount and the target leaves a claim reaching balances bridged afterwards through routes that never touch it. In the submitted PoC, 500 of 501 tokens drained were staged after the interaction. Nothing surfaces delegate state and the user is not notified.

The CEA ATA holds a resting balance by design: `send_universal_tx_to_ua` withdraws against `parsed_cea_ata.amount` rather than the current staged

amount, unconsumed staged tokens are never swept back, and neither staging nor the Push to Solana withdraw path is rate limited.

The user need not knowingly pick a hostile program. A target honest when reviewed and compromised later poisons every user who invokes it from that point on. Signer privilege does not persist across transactions, so users who stopped invoking it beforehand are unaffected.

Assets:

- `execute.rs` [<https://github.com/pushchain/push-chain-gateway-contracts/tree/audit-main/contracts/svm-gateway>]
- `validation.rs` [<https://github.com/pushchain/push-chain-gateway-contracts/tree/audit-main/contracts/svm-gateway>]

Status:

Fixed

Classification

Impact Rate:	3/5
Likelihood Rate:	2/5
Exploitability:	Independent
Complexity:	Complex
Severity:	Medium

Recommendations

Remediation:

Bound the delegate after the CPI: reject a `delegated_amount` exceeding the amount staged in that transaction, or snapshot `delegate` and `delegated_amount` before the CPI and require them unchanged unless the delegation was part of the signed intent. This removes the multiplying property regardless of how funds later reach the ATA, and avoids the blanket `delegate.is_none()` check that would block legitimate integrations.

As a supplementary measure, sweep any unconsumed staged balance from the CEA ATA back to the vault after the CPI returns. This closes the residue route only. It does not cover a withdraw whose recipient is the CEA: that path returns before any dispatch occurs, so a post-CPI sweep never runs on it and the parked balance still lands on a poisoned ATA.

To prevent reassignment, assert `cea.owner == system_program::ID && cea.data_is_empty()` after `invoke_signed`. Structurally, making the CEA program owned prevents `assign` entirely, but the two places that move SOL out of the CEA would need rewriting to direct lamport arithmetic, since a system transfer requires a system owned source, and the CEA would lose the ability to act as a rent payer for target programs.

To protect ATA ownership, assert the CEA ATA `owner` and `close_authority` are unchanged after the CPI.

Off chain and requiring no code change, the target, account list, flags and instruction data are all inside the TSS signed payload, so requests handing CEA authority to unvetted targets can be filtered at signing time.

Resolution:

Fixed in commit `998a837`.

The post-CPI pass was extended to cover CEA-owned SPL token accounts that appear during the CPI, not only those in the pre-CPI snapshot. Canonical CEA ATA addresses are derived from mint accounts collected from `remaining_accounts`, and any such account must exit the CPI with no delegate, zero delegated amount, and no close authority.

The delegate bound was changed from the staged amount to the balance remaining after the CPI. The delegate-identity guard from the previous commit is retained.

Evidences

PoC

Reproduce:

Steps to Reproduce

1. Attacker deploys program `P` that CPIs `spl_token::Approve` on whatever token account it receives, naming itself delegate with `u64::MAX`.
2. Victim sends an execute-mode universal tx targeting `P`, staging a small SPL amount, with the CEA and CEA ATA in the signed account list and the ATA writable.
3. `stage_assets_to_cea` moves the amount to the CEA ATA, then `invoke_signed` (execute.rs:607) calls `P` with the CEA promoted to signer (execute.rs:592).
4. `P` relays `Approve`. Signer privilege carries into the nested CPI, so SPL Token accepts it.
5. Dispatch returns to `Ok(())`. Nothing re-reads the ATA; pre-CPI checks covered mint and owner only (execute.rs:770-775). The delegate persists.
6. Victim later bridges more SPL through ordinary flows. Balances accumulate in the CEA ATA as staging residue or a withdraw-to-CEA. `P` is not involved.
7. Attacker transfers from the CEA ATA as delegate. No gateway involvement, and the allowance survives every future refill.

Results:

PSVMSCDD-9: recovery path verification

```

=== T1 baseline (researcher's claim) ===
  staged via target : 1000000
  parked later      : 500000000
  delegate          : 68GmWjfUEi7Dky4L2Rq2unyXQ4SxQcMV8KfJpYboWcBa
  allowance         : 18446744073709551615
  drained           : 501000000
  NOVEL (never touched target): 500000000
    ✓ T1 baseline: delegate survives and drains a LATER refill (1237ms)

=== T2 (D2): revocation without upgrade ===
  before : delegate=68GmWjfUEi7Dky4L2Rq2unyXQ4SxQcMV8KfJpYboWcBa allowance=18
446744073709551615
  after  : delegate=null allowance=0
  balance preserved: 501000000
  post-revoke sweep BLOCKED: Simulation failed.
    ✓ T2 (D2): execute path relays spl_token Revoke and clears the delegate (
1228ms)

=== T3 (D3): evacuation to vault ===
  CEA ATA before : 501000000
  CEA ATA after  : 0
  vault delta    : 501000000
  delegate still present: 68GmWjfUEi7Dky4L2Rq2unyXQ4SxQcMV8KfJpYboWcBa
  post-evacuation sweep BLOCKED: Simulation failed.
    ✓ T3 (D3): send_universal_tx_to_uea evacuates a poisoned balance (1218ms)

=== T4 (D1): assign ===
  CEA owner now : HDk89xoq8KNwksQHnrZrnguAwGGAUDMd6gv2zy2n1oSi
  stranded SOL  : 2000000000 lamports
  (this is what an upgrade could NOT recover)
  follow-up finalize BLOCKED: AnchorError caused by account: cea_authority. E
rror Code: AccountNotSystemOwned. Error Number: 3011. Error Message: The give
n account is not owned by the system program.
    ✓ T4 (D1): assign bricks every finalize path; measure stranded SOL (410ms
)
  - T5 (D1 proof): patched build recovers SPL from a reassigned CEA
  later staging BLOCKED: AnchorError thrown in programs/universal-gateway/src
/instructions/execute.rs:772. Error Code: InvalidOwner. Error Number: 6008. E
rror Message: Invalid owner.

=== T6 (D4): SetAuthority fails closed ===
  ATA owner now      : 68GmWjfUEi7Dky4L2Rq2unyXQ4SxQcMV8KfJpYboWcBa
  balance at takeover: 1000000
  vault delta        : 0
    ✓ T6 (D4): SetAuthority blocks all later staging; no future refill (809ms
)

=== T7 (D5): concurrent, not exclusive ===
  payee received     : 100000000

```

ATA owner unchanged: **true**
delegate persists : 68GmWjfUEi7Dky4L2Rq2unyXQ4SxQcMV8KfJpYboWcBa
✓ T7 (D5): gateway retains owner authority **while** a delegate is installed
(1610ms)

=== T8 (adverse check on D6) ===

staged : **250000000**
resting **in** CEA ATA: **250000000**
=> withdraw-to-CEA is NOT required to produce a resting balance
✓ T8 (adverse): unconsumed staging residue rests **in** the CEA ATA (407ms)

F-2026-15624 - Excessive Pyth Oracle Staleness Window Enables Rate Limit Bypass - Low

Description:

The `MAX_PRICE_AGE_SECONDS` constant is set to 1 hour, which is significantly more generous than the industry-standard range of 30–90 seconds for on-chain price staleness checks. This value is passed to Pyth's `get_price_no_older_than()` and serves as the sole freshness gate for SOL/USD pricing used in the GAS-route deposit flow.

```
const MAX_PRICE_AGE_SECONDS: u64 = 3_600; // 1 hour
```

The SOL/USD price is used to enforce per-transaction USD caps (`min_cap_universal_tx_usd`, `max_cap_universal_tx_usd`) and the per-block USD budget (`block_usd_cap`) on the instant Gas route. With Pyth's pull-based oracle model, the user controls which price attestation gets posted to the `PriceUpdateV2` account before their transaction. A 1-hour window gives users meaningful ability to select the most favorable price from recent history.

The staleness window creates risk in both directions:

Stale price below market: The USD conversion underreports the true deposit value. Deposits that exceed `max_cap_usd` at current market price pass the cap check using the outdated lower price, allowing more value through the instant route than intended.

Stale price above market: The USD conversion overreports the deposit value. Legitimate deposits that fall within the cap range at current market price are rejected at `min_cap_usd`, blocking valid bridge usage until a fresher price is posted.

The same staleness also distorts the per-slot block USD cap (`check_block_usd_cap`), inflating or deflating the consumed budget relative to actual USD throughput.

Assets:

- pricing.rs [<https://github.com/pushchain/push-chain-gateway-contracts/tree/audit-main/contracts/svm-gateway>]

Status:

Fixed

Classification

Impact Rate:

2/5

Likelihood Rate: 3/5

Exploitability: Independent

Complexity: Simple

Severity: Low

Recommendations

Remediation: Reduce `MAX_PRICE_AGE_SECONDS` to a value aligned with industry standards:

```
const MAX_PRICE_AGE_SECONDS: u64 = 60; // 1 minute
```

Alternatively, make the staleness threshold admin-configurable so that it can be tuned without a program upgrade.

Resolution: Fixed in `d48473e`.

The hardcoded 1-hour Pyth window is replaced by an admin-configurable `config.pyth_max_age_seconds` (default 60s, must be > 0) used in `check_usd_caps`; residual note: `get_sol_price` still hardcodes `3_600` and should take `max_age_seconds` as a parameter to avoid reproducing the same anti-pattern.

[F-2026-15699](#) - set_token_rate_limit Unconditionally Resets Epoch Usage on Threshold Update - Low

Description: Every call to `set_token_rate_limit` unconditionally resets `epoch_usage` to `{ epoch: 0, used: 0 }`, even when updating an already-initialized account. Updating a token threshold mid-epoch silently clears accumulated usage.

This can defeat the operator's intent when tightening limits during abnormal activity, because the token immediately regains fresh capacity up to the new threshold. The issue is admin-triggered and does not let an unprivileged user bypass rate limiting directly.

Assets:

- `admin.rs` [<https://github.com/pushchain/push-chain-gateway-contracts/tree/audit-main/contracts/svm-gateway>]

Status: Fixed

Classification

Impact Rate: 2/5

Likelihood Rate: 2/5

Exploitability: Dependent

Complexity: Simple

Severity: Low

Recommendations

Remediation: Only initialize `epoch_usage` when `init_if_needed` creates the account. Preserve existing usage when updating the threshold. If explicit mid-epoch resets are desired, provide a separate admin-only reset function instead of coupling reset behavior to threshold updates.

Resolution: Fixed in commit `cc3337e`.

`set_token_rate_limit` no longer overwrites `epoch_usage` on update — it relies on Anchor's zero-init for first creation and preserves accumulated usage on subsequent threshold updates.

[F-2026-15701](#) - limit_threshold = 0 Causes Token-Level DoS Instead of Disabling Rate Limiting - Low

Description:

The `set_token_rate_limit` admin setter documents `limit_threshold = 0` as a way to disable rate limiting for a given token, matching EVM behavior:

```
/// @param limit_threshold Max amount per epoch (token's natural units).
///      Set to 0 to disable rate limiting for this token.
pub fn set_token_rate_limit(..., limit_threshold: u128) -> Result<> {
    // Allow limit_threshold = 0 to disable rate limiting (matching EVM behavior)
    let token_rate_limit = &mut ctx.accounts.token_rate_limit;
    token_rate_limit.limit_threshold = limit_threshold;
    // ...
}
```

However, the runtime validation in `validate_token_and_consume_rate_limit` treats `limit_threshold == 0` as "token not supported" and rejects deposits and CEA re-entry for that token:

```
pub fn validate_token_and_consume_rate_limit(...) -> Result<> {
    // ...
    // If limit_threshold == 0, token is not supported
    require!(
        token_rate_limit.limit_threshold > 0,
        GatewayError::NotSupported // Rejects deposit entirely
    );
    // ...
}
```

The documented behavior and the implemented behavior are contradictory. Setting `limit_threshold = 0` does not disable rate limiting; it disables the token for all paths that call `validate_token_and_consume_rate_limit()`, including inbound FUNDS routes and the CEA-to-UEA re-entry path.

- An admin following the documented interface can unintentionally disable a token, causing a token-level denial of service. This is a configuration-driven production footgun, not an unprivileged bypass.
- The mismatch between documentation and implementation creates operational risk, especially if the admin is a multisig or governance process relying on the documented interface.

Assets:

- admin.rs [<https://github.com/pushchain/push-chain-gateway-contracts/tree/audit-main/contracts/svm-gateway>]
- rate_limit.rs [<https://github.com/pushchain/push-chain-gateway-contracts/tree/audit-main/contracts/svm-gateway>]

Status:

Fixed

Classification

Impact Rate: 3/5

Likelihood Rate: 2/5

Exploitability: Dependent

Complexity: Simple

Severity: Low

Recommendations

Remediation: Align the documentation with the implementation, or vice versa. Two options:

Option A - Make 0 truly disable rate limiting (match documentation):

```
pub fn validate_token_and_consume_rate_limit(...) -> Result<()> {
    require!(
        token_rate_limit.token_mint == expected_token_mint,
        GatewayError::InvalidToken
    );

    // limit_threshold == 0 -> token is supported but uncapped
    if token_rate_limit.limit_threshold == 0 {
        return Ok(());
    }

    let epoch_duration = rate_limit_config.epoch_duration_sec;
    if epoch_duration > 0 {
        consume_rate_limit(token_rate_limit, amount, epoch_duration)?;
    }

    Ok(())
}
```

Option B - Update documentation to reflect actual behavior:

```
/// @param limit_threshold Max amount per epoch (token's natural units).  
///      Set to 0 to disable this token (deposits will be rejected).  
///      To allow effectively uncapped deposits, use a very high threshold  
///      and set epoch_duration_sec according to the intended policy.
```

Resolution:

Fixed in commit [4c79861](#).

Push Chain team corrected `limit_threshold = 0` documentation/comments to match runtime behavior: zero now explicitly means token unsupported, not "rate limiting disabled."

[F-2026-15649](#) - Dead Code in send_tx_with_gas_route Zero-Amount Guard - Info

Description:

The `send_tx_with_gas_route()` function is designed exclusively for the `TxType::Gas` and `TxType::GasAndPayload` routes, as enforced by the guard at function entry. Any other `TxType` is rejected before any logic executes.

Despite this, a zero-amount guard matches against `TxType::FundsAndPayload`, which can never be reached within this function.

```
fn send_tx_with_gas_route(..){

    // Function restricted to Gas and GasAndPayload only
    require!(
        matches!(tx_type, TxType::Gas | TxType::GasAndPayload),
        GatewayError::InvalidTxType
    );

    // ... rest of the code

    if gas_amount == 0 {
        require!(
            matches!(tx_type, TxType::GasAndPayload | TxType::FundsAndPayload)
        , // FundsAndPayload is dead
            GatewayError::InvalidAmount
        );
    }

    // ... rest of the code
}
```

The presence of `FundsAndPayload` in a function that explicitly rejects it at entry increases cognitive overhead during review, increases Gas costs, and may indicate an incomplete refactor .

Assets:

- deposit.rs [<https://github.com/pushchain/push-chain-gateway-contracts/tree/audit-main/contracts/svm-gateway>]

Status:

Fixed

Classification

Impact Rate:

1/5

Likelihood Rate: 5/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation: Simplify the guard to reflect the only reachable state:

```
if gas_amount == 0 {  
    require!(tx_type == TxType::GasAndPayload, GatewayError::InvalidAmount);  
}
```

Resolution: Fixed in commit [5750c88](#).

`matches!(tx_type, TxType::GasAndPayload | TxType::FundsAndPayload)` is replaced by zero-gas guard with a direct `tx_type == TxType::GasAndPayload` check.

F-2026-15654 - Misleading Function and Parameter Naming in Instant Route Handler - Info

Description:

The function `send_tx_with_gas_route()` serves as the handler for the protocol's instant route, processing both `TxType::Gas` and `TxType::GasAndPayload` deposits. The function name and its `gas_amount` parameter suggest that native SOL (gas) is always transferred as part of the operation. However, the function contains an explicit code path for `gas_amount == 0` that handles payload-only execution with no SOL transfer, no USD cap enforcement, and no rate limit consumption.

```
fn send_tx_with_gas_route(
    ctx: &mut Context<SendUniversalTx>,
    tx_type: TxType,
    gas_amount: u64,
    payload: &[u8],
    revert_recipient: &Pubkey,
    signature_data: &[u8],
) -> Result<()> {
    // ...
    if gas_amount == 0 {
        emit!(UniversalTx { amount: 0, ... });
        return Ok(());
    }
    // ...
}
```

This naming inconsistency extends to the call site in `route_universal_tx()`, where `native_amount` (the post-fee adjusted SOL value) is passed as `gas_amount`. After protocol fee deduction, this value can legitimately be zero for `GasAndPayload` transactions, creating a disconnect between the parameter name and its actual semantics.

Assets:

- deposit.rs [<https://github.com/pushchain/push-chain-gateway-contracts/tree/audit-main/contracts/svm-gateway>]

Status:

Accepted

Classification

Impact Rate: 1/5

Likelihood Rate: 5/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation: Rename the function and parameter to accurately reflect the full scope of the instant route handler:

```
fn send_tx_with_instant_route(  
    ctx: &mut Context<SendUniversalTx>,  
    tx_type: TxType,  
    sol_amount: u64,  
    payload: &[u8],  
    revert_recipient: &Pubkey,  
    signature_data: &[u8],  
) -> Result<()> {
```

Resolution: **Accepted.**

The Push Chain team accepted the given finding, and no further changes were applied.

F-2026-15705 - Unprotected Initialization Allows First-Caller

Takeover - Info

Description:

The program does not enforce who may perform the first initialization. The `admin` signer in the `Initialize` accounts struct is simply the payer - there is no constraint tying it to the program's upgrade authority, a hardcoded deployer key, or any other trusted identity. As a result, the first caller to invoke `initialize` sets the initial privileged roles and core configuration.

Re-initialization is prevented by Anchor's `init` constraint, so the risk exists only during the deployment-to-initialization window. If deployment and initialization are performed in separate transactions, an external caller could win the first-initialization race and set the initial admin, pauser, TSS, and oracle configuration. This would hand control of the freshly deployed instance to the wrong party. The issue does not remain exploitable after successful initialization.

Assets:

- `initialize.rs` [<https://github.com/pushchain/push-chain-gateway-contracts/tree/audit-main/contracts/svm-gateway>]

Status:

Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 1/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation:

Restrict first initialization to the program's current upgrade authority instead of relying on an atomic deploy-and-initialize procedure. This removes the first-caller race and ensures only the trusted deployer authority can set the initial privileged roles.

```
use anchor_lang::solana_program::bpf_loader_upgradeable::ProgramData;

#[derive(Accounts)]
```

```

pub struct Initialize<'info> {
    #[account(init, seeds = [CONFIG_SEED], bump, payer = admin, space = Config::LEN)]
    pub config: Account<'info, Config>,

    pub program: Program<'info, UniversalGateway>,

    #[account(
        constraint = program.programdata_address()? == Some(program_data.key(
    ))

        @ GatewayError::Unauthorized
    )]
    pub program_data: Account<'info, ProgramData>,

    #[account(
        mut,
        constraint = program_data.upgrade_authority_address == Some(admin.key(
    ))

        @ GatewayError::Unauthorized
    )]
    pub admin: Signer<'info>,

    pub system_program: Program<'info, System>,
}

```

Resolution:

Fixed in commit [7932c97](#).

`initialize(...)` is now gated to the program's upgrade authority by requiring `program / ProgramData` accounts and enforcing `program_data.upgrade_authority_address == admin`.

[F-2026-15754](#) - Redundant Payload Validation in Funds Route

Handler - Info

Description:

The `send_tx_with_funds_route` function validates payload presence against `tx_type`, but these checks are unreachable because the upstream `fetch_tx_type` function already determines the type from the same payload state.

```
if tx_type == TxType::Funds {
    require!(req.payload.is_empty(), GatewayError::InvalidInput);
} else {
    require!(!req.payload.is_empty(), GatewayError::InvalidInput);
}
```

The `fetch_tx_type` function infers the transaction type directly from `req.payload.is_empty()`:

```
fn fetch_tx_type(req: &UniversalTxRequest, native_amount: u64) -> Result<TxType> {
    let has_payload = !req.payload.is_empty();
    let has_funds = req.amount > 0;
    // ...
    if has_payload {
        return Ok(TxType::FundsAndPayload); // payload non-empty → FundsAndPayload
    }
    Ok(TxType::Funds) // payload empty → Funds
}
```

Since `tx_type` is derived from the payload content before `send_tx_with_funds_route` is called, the following invariants are guaranteed at entry:

- `tx_type == Funds` implies `req.payload.is_empty() == true`
- `tx_type == FundsAndPayload` implies `req.payload.is_empty() == false`

Both `require!` branches will always pass. The error paths are dead code.

The gas route handler (`send_tx_with_gas_route`) had equivalent validation that has been commented out, creating an inconsistency between the two route handlers. Neither set of checks has any effect on program behavior.

Status:

Fixed

Classification

Impact Rate:	1/5
Likelihood Rate:	5/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Info

Recommendations

Remediation: Remove the redundant checks from `send_tx_with_funds_route` to match the gas route, and removed commented parts in `send_tx_with_gas_route` for uniform defense-in-depth style.

Resolution: **Fixed in commit** `c771071`.

The dead payload-validation `require!`s were removed from `send_tx_with_funds_route` and the matching commented-out block was removed from `send_tx_with_gas_route`, restoring uniform style across both route handlers; cross-checked against `fetch_tx_type` to confirm the removed checks were genuinely unreachable.

F-2026-15756 - Unbounded Protocol Fee - Info

Description:

The `set_protocol_fee()` function accepts any `u64` value as the protocol fee with no upper bound or sanity check:

```
pub fn set_protocol_fee(ctx: Context<FeeVaultAdminAction>, fee_lamports: u64)
-> Result<> {
    ctx.accounts.fee_vault.bump = ctx.bumps.fee_vault;
    ctx.accounts.fee_vault.protocol_fee_lamports = fee_lamports;
    emit!(ProtocolFeeUpdated { new_fee_lamports: fee_lamports });
    Ok(())
}
```

The fee is deducted from the user's `native_amount` before any routing logic runs:

```
fn collect_protocol_fee(ctx: &mut Context<SendUniversalTx>, native_amount: u64) -> Result<u64> {
    let fee_lamports = ctx.accounts.fee_vault.protocol_fee_lamports;
    if fee_lamports == 0 {
        return Ok(native_amount);
    }
    require!(
        native_amount >= fee_lamports,
        GatewayError::InsufficientProtocolFee
    );
    // Transfer fee from user -> fee_vault
    // ...
    let adjusted_native_amount = native_amount - fee_lamports;
    // ...
    Ok(adjusted_native_amount)
}
```

Without an upper bound, a compromised or negligent admin can set the protocol fee to absorb the majority of user deposits. The fee change takes effect immediately with no delay, timelock, or on-chain notification prior to execution. Users have no mechanism to specify a maximum acceptable fee, so any in-flight deposit transactions will silently have the new fee deducted regardless of what the user expected at submission time.

Even without malicious intent, a routine admin fee update can accidentally front-run pending user transactions within the same slot. If the admin's `set_protocol_fee()` instruction is ordered before a user's `send_universal_tx()`

in the same block, the user's deposit is processed under the new fee without their knowledge or consent. The user submits a transaction expecting the full `native_amount` to be bridged, but a significant portion is silently redirected to the fee vault, resulting in a bridge deposit far smaller than intended.

Assets:

- lib.rs [<https://github.com/pushchain/push-chain-gateway-contracts/tree/audit-main/contracts/svm-gateway>]

Status:

Mitigated

Classification**Impact Rate:**

3/5

Likelihood Rate:

1/5

Exploitability:

Dependent

Complexity:

Simple

Severity:

Info

Recommendations**Remediation:**

Introduce a maximum fee cap to bound the protocol fee and add a `max_fee` parameter to `send_universal_tx` so users can reject transactions where the fee exceeds their expectation at submission time.

Resolution:

Mitigated in commit `7ca330f`.

Caps `set_protocol_fee` at `MAX_PROTOCOL_FEE_LAMPORTS = 2_000_000` lamports (0.002 SOL), which closes the unbounded-fee concern but does not address the related no-slippage / no-timelock front-running scenario.

F-2026-15757 - Lack of Two-Step Admin Transfer - Info

Description:

The `set_authorities()` function transfers admin and pauser roles in a single step without requiring the new authority to confirm acceptance:

```
pub fn set_authorities(
    ctx: Context<SetAuthoritiesAction>,
    new_admin: Option<Pubkey>,
    new_pauser: Option<Pubkey>,
) -> Result<()> {
    require!(
        new_admin.is_some() || new_pauser.is_some(),
        GatewayError::InvalidInput
    );
    let config = &mut ctx.accounts.config;
    if let Some(next) = new_admin {
        require!(next != Pubkey::default(), GatewayError::ZeroAddress);
        config.admin = next;
    }
    if let Some(next) = new_pauser {
        require!(next != Pubkey::default(), GatewayError::ZeroAddress);
        config.pauser = next;
    }
    Ok(())
}
```

The only validation is a zero-address check. If the admin mistakenly provides an incorrect pubkey — a typo, a key from the wrong environment, or a key whose private key is unavailable — admin access is permanently transferred to an uncontrollable address in a single transaction.

Assets:

- admin.rs [<https://github.com/pushchain/push-chain-gateway-contracts/tree/audit-main/contracts/svm-gateway>]

Status:

Fixed

Classification

Impact Rate: 3/5

Likelihood Rate: 1/5

Exploitability: Dependent

Complexity: Simple

Severity: Info

Recommendations

Remediation: Implement a two-step transfer pattern where the proposed authority must explicitly accept the role:

```
pub fn propose_admin(ctx: Context<AdminAction>, proposed: Pubkey) -> Result<()> {
    require!(proposed != Pubkey::default(), GatewayError::ZeroAddress);
    ctx.accounts.config.pending_admin = Some(proposed);
    Ok(())
}

pub fn accept_admin(ctx: Context<AcceptAdmin>) -> Result<()> {
    let config = &mut ctx.accounts.config;
    require!(
        config.pending_admin == Some(ctx.accounts.new_admin.key()),
        GatewayError::Unauthorized
    );
    config.admin = ctx.accounts.new_admin.key();
    config.pending_admin = None;
    Ok(())
}
```

Resolution: Fixed in commit [478a7dd](#).

The fix replaces the single-step `set_authorities` with a two-step flow — `propose_authorities` writes the new admin/pauser into new `pending_admin` / `pending_pauser` fields on `Config`, and the live roles only update once the proposed key itself signs `accept_admin` / `accept_pauser`, preventing accidental transfer to an uncontrollable address.

[F-2026-15930](#) - Redundant Oracle Price Calculation in Gas Route

Deposit - Info

Description:

The `send_tx_with_gas_route()` function calls `check_usd_caps()` followed by `calculate_sol_price()` and `calculate_usd_amount()`, performing the same Pyth oracle deserialization and USD conversion twice.

```
check_usd_caps(&ctx.accounts.config, gas_amount, &ctx.accounts.price_update);  
  
let price_data = calculate_sol_price(&ctx.accounts.price_update)?;  
let usd_amount = calculate_usd_amount(gas_amount, &price_data)?;  
check_block_usd_cap(&mut ctx.accounts.rate_limit_config, usd_amount)?;
```

Internally, `check_usd_caps` already performs both operations:

```
pub fn check_usd_caps(  
    config: &Config,  
    lamports: u64,  
    price_update: &Account<PriceUpdateV2>,  
) -> Result<()> {  
    let price_data = calculate_sol_price(price_update)?; // ← first call  
  
    if config.pyth_confidence_threshold > 0 {  
        require!(  
            price_data.confidence <= config.pyth_confidence_threshold,  
            GatewayError::InvalidPrice  
        );  
    }  
  
    let usd_amount = calculate_usd_amount(lamports, &price_data)?; // ← first conversion  
  
    require!(usd_amount >= config.min_cap_universal_tx_usd, GatewayError::BelowMinCap);  
    require!(usd_amount <= config.max_cap_universal_tx_usd, GatewayError::AboveMaxCap);  
    Ok(())  
}
```

The `calculate_sol_price()` function parses the Pyth `PriceUpdateV2` account, performs feed ID validation, staleness checks via `get_price_no_older_than()`, and positive price validation. Running this twice per deposit wastes compute units — a meaningful overhead for a bridge instruction that relayers pay for on every gas route deposit.

Assets:

- deposit.rs [<https://github.com/pushchain/push-chain-gateway-contracts/tree/audit-main/contracts/svm-gateway>]

Status:

Fixed

Classification

Impact Rate:	1/5
Likelihood Rate:	5/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Info

Recommendations

Remediation: Refactor `check_usd_caps` to return the computed `usd_amount`, eliminating the redundant second call:

```
pub fn check_usd_caps(
    config: &Config,
    lamports: u64,
    price_update: &Account<PriceUpdateV2>,
) -> Result<u128> {
    let price_data = calculate_sol_price(price_update)?;
    if config.pyth_confidence_threshold > 0 {
        require!(price_data.confidence <= config.pyth_confidence_threshold, GatewayError::InvalidPrice);
    }
    let usd_amount = calculate_usd_amount(lamports, &price_data)?;
    require!(usd_amount >= config.min_cap_universal_tx_usd, GatewayError::BelowMinCap);
    require!(usd_amount <= config.max_cap_universal_tx_usd, GatewayError::AboveMaxCap);
    Ok(usd_amount)
}
```

Then in the gas route:

```
let usd_amount = check_usd_caps(&ctx.accounts.config, gas_amount, &ctx.accounts.price_update)?;
check_block_usd_cap(&mut ctx.accounts.rate_limit_config, usd_amount)?;
```

This removes two function calls, one Pyth account deserialization, and one fixed-point multiplication per gas route deposit.

Resolution:

Fixed in commit `7cb6ac2`.

`check_usd_caps` now returns `Result<u128>` with the computed `usd_amount`, eliminating the redundant `calculate_sol_price` and `calculate_usd_amount` calls in the gas route deposit path. One Pyth oracle deserialization and one fixed-point multiplication removed per gas route deposit.

[F-2026-15948](#) - PDA Bumps Re-derived Instead of Using Stored Values in Finalize Instruction - Info

Description:

The `FinalizeUniversalTx` account struct uses bare `bump` constraints for `config`, `cea_authority`, and `tss_pda`, causing Anchor to re-derive the canonical bump by iterating from 255 downward on every invocation. In contrast, `RevertUniversalTx` and `RescueFunds` use stored bumps, avoiding the derivation overhead.

```
pub struct FinalizeUniversalTx<'info> {  
  
    // ...  
  
    #[account(  
        seeds = [b"config"],  
        bump,  
    )]  
    pub config: Account<'info, Config>,  
    #[account(  
        mut,  
        seeds = [CEA_SEED, push_account.as_ref()],  
        bump,  
    )]  
    pub cea_authority: SystemAccount<'info>,  
    #[account(  
        mut,  
        seeds = [TSS_SEED],  
        bump,  
    )]  
    pub tss_pda: Account<'info, TssPda>,  
  
    // ...  
  
}
```

Revert uses stored bumps:

```
pub struct RevertUniversalTx<'info> {  
  
    // ...  
  
    #[account(  
        seeds = [CONFIG_SEED],  
        bump = config.bump,           // stored bump – no iteration  
    )]  
  
}
```

```
pub config: Account<'info, Config>,
#[account(mut, seeds = [VAULT_SEED], bump = config.vault_bump)]
pub vault: UncheckedAccount<'info>,
#[account(mut, seeds = [TSS_SEED], bump = tss_pda.bump)]
pub tss_pda: Account<'info, TssPda>,
// ...
```

Anchor's `find_program_address` iterates from bump value 255 downward until it finds a valid PDA. Each iteration costs approximately 1,500 compute units. While the canonical bump is statistically found within the first few attempts, the overhead is unnecessary when the bump is already stored in the account's on-chain data.

Each finalize transaction wastes approximately 3,000 compute units on unnecessary PDA bump re-derivation for `config` and `tss_pda`. For a bridge processing high transaction volume, this accumulates into higher aggregate compute consumption and increased priority fee costs for relayers.

Assets:

- `execute.rs` [<https://github.com/pushchain/push-chain-gateway-contracts/tree/audit-main/contracts/svm-gateway>]

Status:

Fixed

Classification

Impact Rate:	1/5
Likelihood Rate:	5/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Info

Recommendations

Remediation:

Use stored bumps for `config` and `tss_pda` in the `FinalizeUniversalTx` account struct, consistent with the pattern used by `RevertUniversalTx` and `RescueFunds`:

```
#[account(
  seeds = [CONFIG_SEED],
  bump = config.bump,
)]
pub config: Account<'info, Config>,
```

```
#[account(  
    mut,  
    seeds = [TSS_SEED],  
    bump = tss_pda.bump,  
)]  
pub tss_pda: Account<'info, TssPda>,
```

The `cea_authority` must remain with bare bump since it is a SystemAccount with no stored state.

Resolution:

Fixed in commit `ee8475f`.

Stored bumps are now used for `config` (`config.bump`) and `tss_pda` (`tss_pda.bump`) in the `FinalizeUniversalTx` account struct, eliminating unnecessary bump re-derivation on every finalize call. `cea_authority` retains bare `bump` as its value is not persisted on-chain, making re-derivation unavoidable on that account.

[F-2026-15957](#) - Missing Mint Authority Validation in `set_token_rate_limit` Allows Whitelisting Tokens with Unsafe Authorities - Info

Description: The `set_token_rate_limit` path uses `UncheckedAccount` for `token_mint`, so the on-chain code never inspects the mint's `freeze_authority` or `mint_authority` fields at whitelist time.

```
/// CHECK: Token mint address
pub token_mint: UncheckedAccount<'info>,
```

Because the whitelist path does not inspect `mint_authority` or `freeze_authority`, the protocol can whitelist tokens whose issuer retains privileged control.

- A token with active `freeze_authority` can have vault token accounts frozen, disrupting withdraw, revert, or rescue flows.
- A token with active `mint_authority` can be inflated after listing, weakening the bridge's collateral assumptions for that asset.

Status: Fixed

Classification

Impact Rate:	1/5
Likelihood Rate:	1/5
Exploitability:	Dependent
Complexity:	Simple
Severity:	Info

Recommendations

Remediation: Change `token_mint` to `Account<'info, Mint>` so the setter can inspect authority fields at whitelist time. Three approaches, in order of resilience:

Option A - On-chain trusted token registry (most resilient):

Maintain a PDA-based registry of trusted token mints whose issuers are known to hold `freeze_authority` or `mint_authority` (e.g., USDC, USDT). The setter checks the registry and branches accordingly:

```
let mint = &ctx.accounts.token_mint; // now Account<Mint>

if !is_trusted_token(&ctx.accounts.trusted_registry, mint.key()) {
    require!(mint.freeze_authority.is_none(), GatewayError::InvalidMint);
    require!(mint.mint_authority.is_none(), GatewayError::InvalidMint);
}
```

Option B - Boolean flag (minimal change):

Add a `trusted_authority` parameter to `set_token_rate_limit`. Admin explicitly opts in for known issuers:

```
pub fn set_token_rate_limit(
    ctx: Context<TokenRateLimitAction>,
    limit_threshold: u128,
    trusted_authority: bool,
) -> Result<()> {
    let mint = &ctx.accounts.token_mint; // now Account<Mint>

    if !trusted_authority {
        require!(mint.freeze_authority.is_none(), GatewayError::InvalidMint);
        require!(mint.mint_authority.is_none(), GatewayError::InvalidMint);
    }

    // ... existing logic
}
```

Option C - Off-chain validation with documented operational assumption:

If off-chain validation is considered sufficient and both token types are required to be supported, document the operational expectation that the admin must evaluate the trust model of each token's issuer before whitelisting.

The risks outlined in the description section should be explicitly acknowledged in the protocol's operational runbook and token onboarding process.

Resolution:

Fixed in commit `b85871c`,

Now `set_token_rate_limit` validates SPL mint/freeze authorities on-chain and requires explicit acknowledgment flags before enabling a non-zero threshold for authority-bearing mints.

F-2026-18981 - Rate Limiting Scope Is Not Documented in the Threat Model - Info

Description:

The gateway implements a per block USD cap (`check_block_usd_cap`) and a per mint epoch threshold (`validate_token_and_consume_rate_limit`).

Enumerating every call site shows they are applied in four places only:

Path	Direction	Rate limited
<code>deposit</code> SOL and SPL	Solana into the bridge	Yes
<code>send_universal_tx_to_uea</code>	CEA back into the vault	Yes
<code>stage_assets_to_cea</code>	Vault out to the user CEA	No
<code>internal_withdraw</code>	CEA out to the recipient	No
<code>revert_universal_tx</code>	Vault out to the revert recipient	No

No Solana side release path is throughput gated.

The client has confirmed this is intentional: the limits are inbound PRC20 mint caps, outbound is authorised per operation by TSS rather than throughput gated, and the emergency control on the release side is `pause` .

No security impact. The asymmetry is by design and a compensating control exists.

The boundary itself is not stated in the code or the threat model, and the account layout points the other way: the finalize instruction declares optional rate limit accounts on paths that never consume them, which reads as though release flows are gated. A later change to a release path could then be made on the assumption that a throughput ceiling already applies, or an intentional design choice could be mistaken for an oversight and 'fixed' into one.

Assets:

- `execute.rs` [<https://github.com/pushchain/push-chain-gateway-contracts/tree/audit-main/contracts/svm-gateway>]
- `deposit.rs` [<https://github.com/pushchain/push-chain-gateway-contracts/tree/audit-main/contracts/svm-gateway>]
- `withdraw.rs` [<https://github.com/pushchain/push-chain-gateway-contracts/tree/audit-main/contracts/svm-gateway>]
- `revert.rs` [<https://github.com/pushchain/push-chain-gateway-contracts/tree/audit-main/contracts/svm-gateway>]

Status:

Fixed

Classification

Impact Rate:	1/5
Likelihood Rate:	5/5
Exploitability:	Dependent
Complexity:	Simple
Severity:	Info

Recommendations

Remediation:	Document in the threat model that the rate limits bound inbound PRC20 minting only, that outbound releases are authorised per operation by TSS rather than throughput gated, and that <code>pause</code> is the release side emergency control. No code change required.
Resolution:	Fixed in commit <code>d637de5</code>. THREAT_MODEL.md entry 17 records the inbound-only rate limiting scope, with doc comments on the optional rate limit accounts pointing back to it.

Disclaimers

Hacken Disclaimer

The smart contracts given for audit have been analyzed based on best industry practices at the time of the writing of this report, with cybersecurity vulnerabilities and issues in smart contract source code, the details of which are disclosed in this report (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

The report contains no statements or warranties on the identification of all vulnerabilities and security of the code. The report covers the code submitted and reviewed, so it may not be relevant after any modifications. Do not consider this report as a final and sufficient assessment regarding the utility and safety of the code, bug-free status, or any other contract statements.

While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only — we recommend proceeding with several independent audits and a public bug bounty program to ensure the security of smart contracts.

English is the original language of the report. The Consultant is not responsible for the correctness of the translated versions.

As part of Hacken's ongoing quality assurance process, we may conduct re-audits of select projects. These re-audits are performed independently from the original audit and are intended solely for internal quality control and improvement. Updated reports resulting from such re-audits will be shared privately with the respective clients and may be published on the Hacken website only with their explicit consent.

The sole authoritative source for finalized and most up-to-date versions of all reports remains the Audits section at <https://hacken.io/audits/>.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have vulnerabilities that can lead to hacks. Thus, the Consultant cannot guarantee the explicit security of the audited smart contracts.



Appendix 1. Definitions

Severities

When auditing smart contracts, Hacken is using a risk-based approach that considers **Likelihood**, **Impact**, **Exploitability** and **Complexity** metrics to evaluate findings and score severities.

Reference on how risk scoring is done is available through the repository in our Github organization:

[hknio/severity-formula](https://github.com/hacken/severity-formula)

Severity	Description
Critical	Critical vulnerabilities are usually straightforward to exploit and can lead to the loss of user funds or contract state manipulation.
High	High vulnerabilities are usually harder to exploit, requiring specific conditions, or have a more limited scope, but can still lead to the loss of user funds or contract state manipulation.
Medium	Medium vulnerabilities are usually limited to state manipulations and, in most cases, cannot lead to asset loss. Contradictions and requirements violations. Major deviations from best practices are also in this category.
Low	Major deviations from best practices or major Gas inefficiency. These issues will not have a significant impact on code execution.

Potential Risks

The "Potential Risks" section identifies issues that are not direct security vulnerabilities but could still affect the project's performance, reliability, or user trust. These risks arise from design choices, architectural decisions, or operational practices that, while not immediately exploitable, may lead to problems under certain conditions. Additionally, potential risks can impact the quality of the audit itself, as they may involve external factors or components beyond the scope of the audit, leading to incomplete assessments or oversight of key areas. This section aims to provide a broader perspective on factors that could affect the project's long-term security, functionality, and the comprehensiveness of the audit findings.

Appendix 2. Scope

The scope of the project includes the following smart contracts from the provided repository:

Scope Details	
Repository	https://github.com/pushchain/push-chain-gateway-contracts/tree/audit-main/contracts/svm-gateway
Initial Commit	93db3c8e66cc1399ee74fc58339440e78321ae06
Final Commit	d637de5b4bf4aa76837f9084c676817443507f94
Whitepaper	N/A
Requirements	https://www.notion.so/pushprotocol/SVM_Crucial_Checks-335188aea7f480bebd07eb8a8e592f6?source=copy_link
Technical Requirements	contracts/svm-gateway/README.md

Asset	Type
admin.rs [https://github.com/pushchain/push-chain-gateway-contracts/tree/audit-main/contracts/svm-gateway]	Smart Contract
deposit.rs [https://github.com/pushchain/push-chain-gateway-contracts/tree/audit-main/contracts/svm-gateway]	Smart Contract
encoding.rs [https://github.com/pushchain/push-chain-gateway-contracts/tree/audit-main/contracts/svm-gateway]	Smart Contract
errors.rs [https://github.com/pushchain/push-chain-gateway-contracts/tree/audit-main/contracts/svm-gateway]	Smart Contract
execute.rs [https://github.com/pushchain/push-chain-gateway-contracts/tree/audit-main/contracts/svm-gateway]	Smart Contract
initialize.rs [https://github.com/pushchain/push-chain-gateway-contracts/tree/audit-main/contracts/svm-gateway]	Smart Contract
lib.rs [https://github.com/pushchain/push-chain-gateway-contracts/tree/audit-main/contracts/svm-gateway]	Smart Contract
mod.rs [https://github.com/pushchain/push-chain-gateway-contracts/tree/audit-main/contracts/svm-gateway]	Smart Contract
mod.rs [https://github.com/pushchain/push-chain-gateway-contracts/tree/audit-main/contracts/svm-gateway]	Smart Contract
pricing.rs [https://github.com/pushchain/push-chain-gateway-contracts/tree/audit-main/contracts/svm-gateway]	Smart Contract
rate_limit.rs [https://github.com/pushchain/push-chain-gateway-contracts/tree/audit-main/contracts/svm-gateway]	Smart Contract
rescue.rs [https://github.com/pushchain/push-chain-gateway-contracts/tree/audit-main/contracts/svm-gateway]	Smart Contract
revert.rs [https://github.com/pushchain/push-chain-gateway-contracts/tree/audit-main/contracts/svm-gateway]	Smart Contract

Asset	Type
state.rs [https://github.com/pushchain/push-chain-gateway-contracts/tree/audit-main/contracts/svm-gateway]	Smart Contract
transfers.rs [https://github.com/pushchain/push-chain-gateway-contracts/tree/audit-main/contracts/svm-gateway]	Smart Contract
tss.rs [https://github.com/pushchain/push-chain-gateway-contracts/tree/audit-main/contracts/svm-gateway]	Smart Contract
validation.rs [https://github.com/pushchain/push-chain-gateway-contracts/tree/audit-main/contracts/svm-gateway]	Smart Contract
withdraw.rs [https://github.com/pushchain/push-chain-gateway-contracts/tree/audit-main/contracts/svm-gateway]	Smart Contract

Appendix 3. Additional Valuables

Additional Recommendations

The smart contracts in the scope of this audit could benefit from the introduction of automatic emergency actions for critical activities, such as unauthorized operations like ownership changes or proxy upgrades, as well as unexpected fund manipulations, including large withdrawals or minting events. Adding such mechanisms would enable the protocol to react automatically to unusual activity, ensuring that the contract remains secure and functions as intended.

To improve functionality, these emergency actions could be designed to trigger under specific conditions, such as:

- Detecting changes to ownership or critical permissions.
- Monitoring large or unexpected transactions and minting events.
- Pausing operations when irregularities are identified.

These enhancements would provide an added layer of security, making the contract more robust and better equipped to handle unexpected situations while maintaining smooth operations.

Frameworks and Methodologies

This security assessment was conducted in alignment with recognised penetration testing standards, methodologies and guidelines, including the [NIST SP 800-115 – Technical Guide to Information Security Testing and Assessment](#), and the [Penetration Testing Execution Standard \(PTES\)](#). These assets provide a structured foundation for planning, executing, and documenting technical evaluations such as vulnerability assessments, exploitation activities, and security code reviews. Hacken's internal penetration testing methodology extends these principles to Web2 and Web3 environments to ensure consistency, repeatability, and verifiable outcomes.